

PYTHON ESSENTIALS CAPSTONE PROJECT REPORT

SMALL FACTORY MANAGEMENT SYSTEM

A CLI-Based Industrial Operations, Inventory Tracking, and Floor Machine Audit Application
developed using Core Python Paradigms.

SUBMITTED BY

Student Name: **Harsh Vijay**
Registration No: **26BCE10808**
Programme: **B.Tech Computer Science
& Engineering**
Academic Year: **2026 – 2027**

SUBMITTED TO

Faculty Name: **Dr. Priyadharshini B.**
Department: **Computer Science &
Engineering**
Course Title: **Python Essentials-
Evaluated Project**
Portal Reference: **Vityarthi E-Learning Portal**

VELLORE INSTITUTE OF TECHNOLOGY (VIT), BHOPAL

Kotri Kalan, Ashta, Near Indore Road, Bhopal, Madhya Pradesh – 466114

1. Introduction

The **Small Factory Management System** is a high-efficiency, menu-driven Python application developed to streamline the daily operational lifecycle of small-scale manufacturing units. The system serves as a unified digital interface capable of tracking raw material and assembled inventory, categorizing warehouse stock, logging batch production updates, and performing automated daily machinery operational audits.

Designed as part of the *Python Essentials* curriculum on the Vityarthi portal, the project demonstrates core procedural and functional paradigms including dictionary-based records, list-based collections, tuples, formatted screen telemetry, input validation loops, and dynamic operational health indicators.

2. Problem Statement

Small assembly lines and job-shop production floors frequently depend on physical ledgers or fragmented spreadsheet files to log stock receipts and dispatch records. This conventional approach leads to critical operational bottlenecks:

- **Inventory Discrepancies:** Lack of real-time stock deductions during manufacturing cycles leads to unexpected shortages and stockouts.
- **Unscheduled Downtime:** Floor machinery and conveyors lack unified digital shift logs, causing delayed incident reporting and reduced overall factory efficiency.
- **Manual Ledger Latency:** Searching across diverse commodity categories (Raw materials vs. Finished sub-assemblies) manually is time-consuming and error-prone.

3. Functional Requirements

The functional requirements define all distinct functional capabilities implemented within the program:

[1] Add New Product: Prompts the floor supervisor for Product ID, Product Name, Category (e.g., Raw, Assembly), Initial Stock Units, and Unit Price (INR). Stores record with formatted validation.

[2] View All Inventory: Iterates across all stored factory stock records, generating a clean columnar readout of ID, Product Name, Current Available Quantity, and Price.

[3] View Registered Categories: Gathers unique categories present across inventory data using collection operations, aiding quick supply-chain classification.

[4] Update Stock After Production: Adjusts inventory count by adding finished goods produced or decrementing raw material batches consumed during a shift.

[5] Run Floor Machine Audit: Executes a comprehensive telemetry scan of all critical plant machines (Assembly Conveyor, Automated Paint Rig, Hydraulic Press, Packaging Line). Produces an immediate status list (OPERATIONAL vs. FAULT DETECTED), tally of online units, and calculated Overall Plant Efficiency percentage.

[6] Save and Exit: Finalizes system state, executes terminal flushing, and gracefully terminates the session with an exit handshake.

4. Non-Functional Requirements

Non-functional criteria govern the quality, robustness, and execution standards under which the factory terminal operates:

Usability & Ergonomics

High-contrast, amber-monochrome terminal interface designed specifically for industrial ambient conditions and glanceable readability on floor terminals.

Computational Efficiency

Constant-time and linear lookups $O(1)$ to $O(N)$ ensure instant responsiveness during shift changes without latency.

Data Reliability

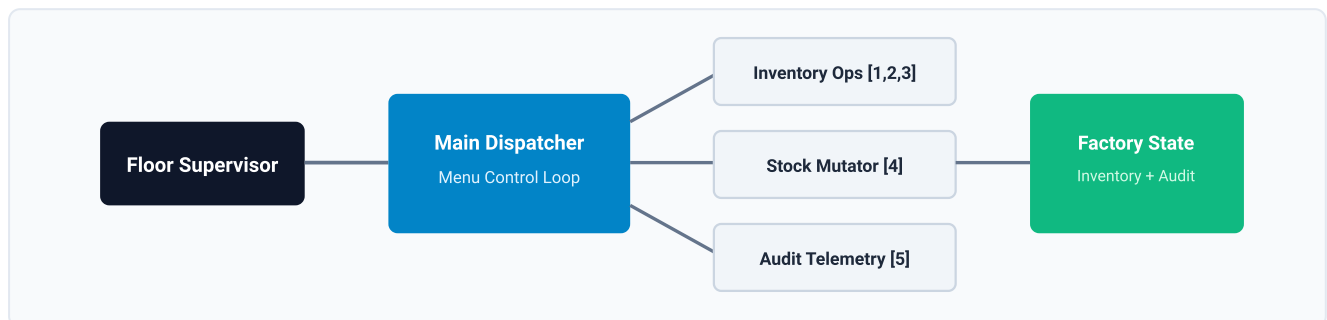
Guarantees strict type casting for integers (stock units) and floats (unit prices) to eliminate arithmetic corruption in financial calculations.

Portability & Footprint

Uses standard Python 3 libraries with zero external heavy dependencies, ensuring compatibility with lightweight Linux boards (e.g., Raspberry Pi) on the plant floor.

5. System Architecture

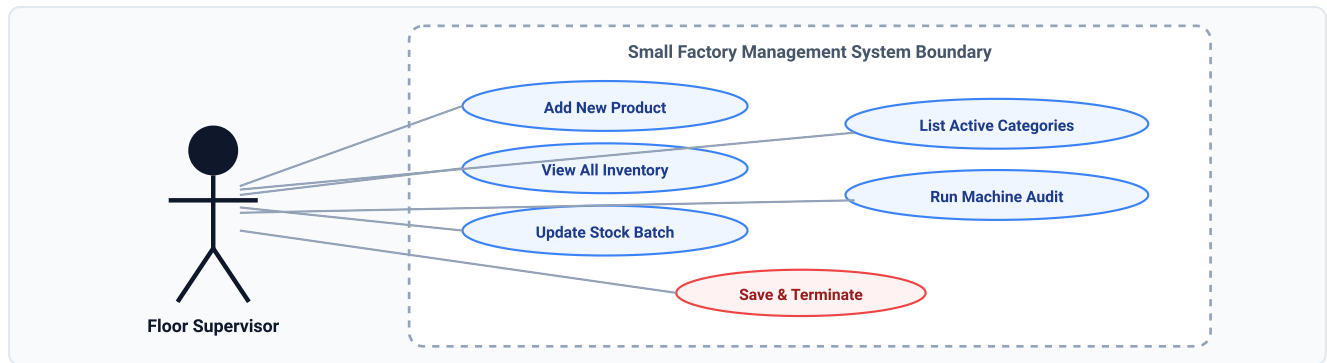
The project adheres to a modular single-program procedural paradigm. Control flow is driven by a main interactive dispatch loop that intercepts operator keystrokes and routes commands to discrete functional sub-modules, interacting with unified in-memory storage.



6. Design Diagrams

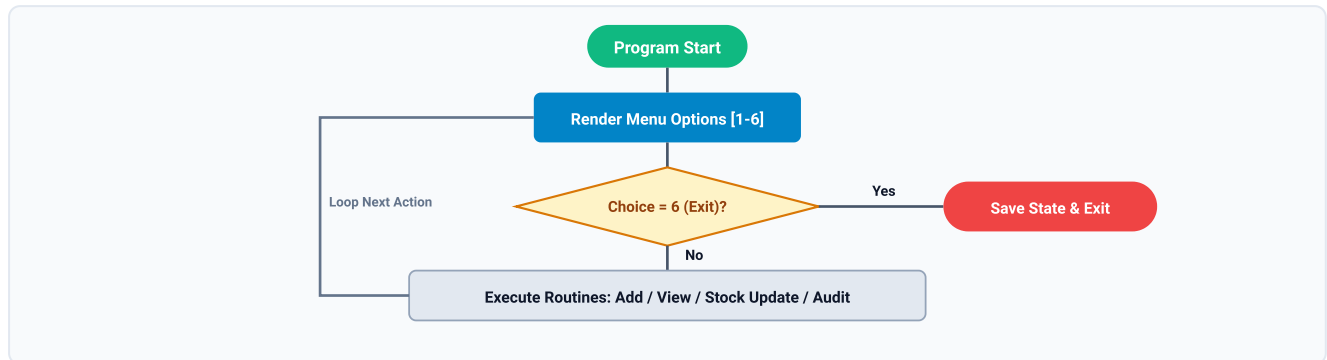
6.1 Use Case Diagram

The Use Case diagram illustrates the behavioral interface between the Factory Supervisor and the core functional modules of the system.



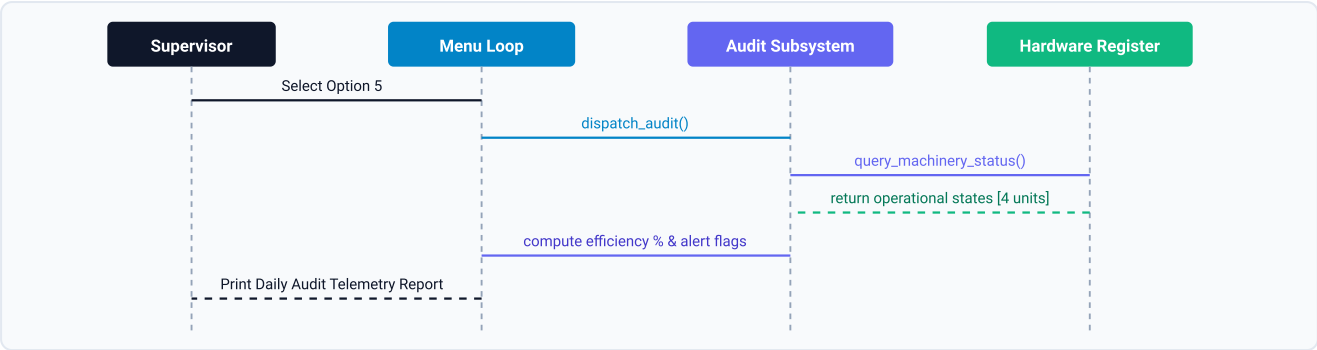
6.2 Workflow & Operational Flowchart

The workflow represents the decision loop from terminal initialization to continuous selection dispatch and program completion.



6.3 Sequence Diagram: Daily Machine Audit Routine

The sequence diagram details the function call timeline when the operator invokes Option 5 ("Run Floor Machine Audit") to assess assembly hardware integrity.



6.4 Component & Entity Relationship Model

The system models physical industrial entities into structured computational records stored in modular memory structures.

ENTITY: INVENTORY_PRODUCT		ENTITY: FACTORY_MACHINE	
product_id	Integer / String (Unique Key)	machine_id	String (e.g. M-01, M-02)
product_name	String (e.g. Cotton, Steel Rivets)	machine_name	String (e.g. Assembly Conveyor)
category	String (Raw, Assembly, Packaging)	current_status	String ("OPERATIONAL" / "FAULT DETECTED")
stock_units	Integer (Current Available Qty)	audit_timestamp	Date / Time String
unit_price	Float (Valuation in INR)	station_role	String (Fabrication, Paint, Packaging)

7. Design Decisions & Rationale

Choice of Core Python:

Python was selected due to its expressive syntax, concise data structures, and rapid development capabilities. In an academic and industrial context, implementing factory workflows without heavy framework dependencies reinforces solid algorithmic thinking and deep command over native constructs.

Dictionary vs. List Representation:

While early prototypes utilized flat tuples, the inventory catalog was designed using dictionary-backed structures where each product is keyed by its unique `product_id`. This enables immediate lookups, modifications, and deletions without scanning entire datasets.

Industrial Telemetry Console UX:

Unlike ordinary academic terminal exercises, the UI incorporates structured divider rules, strict tag formatting ([ALERT], [OK]), and summary metrics (Machines Online: 3 / 4, Overall Efficiency: 75%). This aligns with Supervisory Control and Data Acquisition (SCADA) display principles.

8. Implementation Details

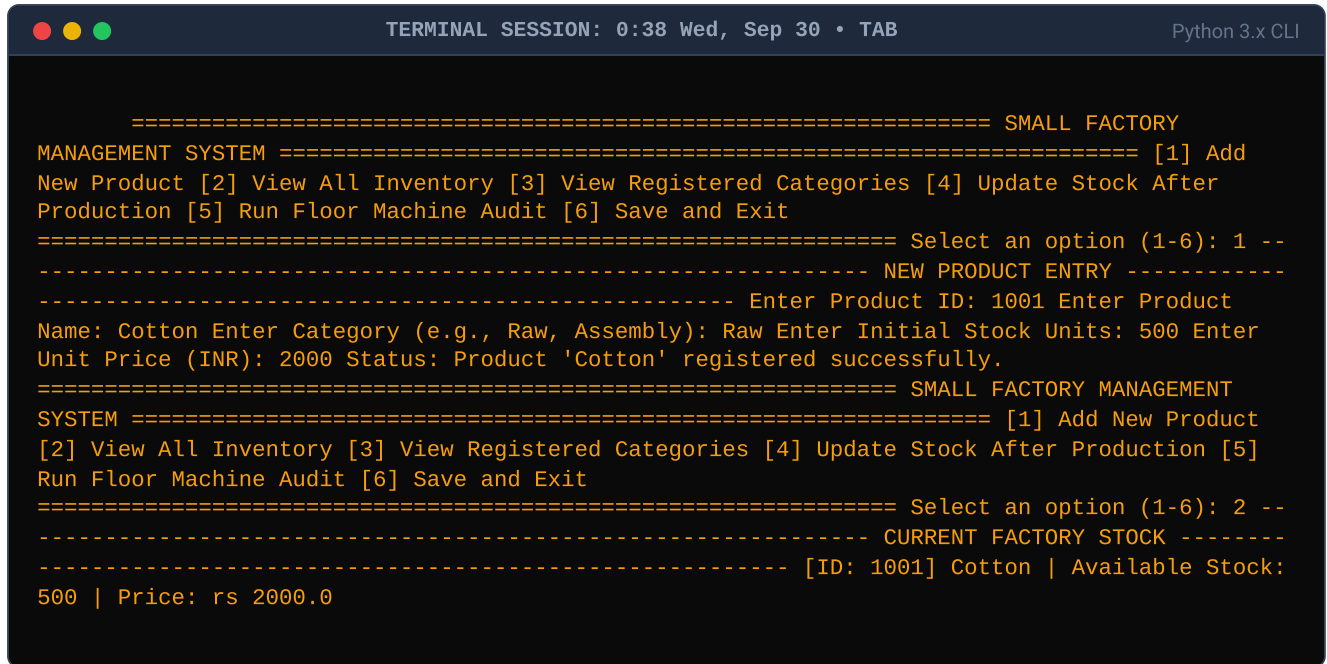
Representative code implementations powering critical operational routines:

```
def run_floor_machine_audit(machines): total = len(machines) online_count = sum(1 for m
in machines if m['status'] == 'OPERATIONAL') efficiency = (online_count / total) * 100 if
total > 0 else 0 print("=" * 60) print(f"DAILY AUDIT REPORT: {datetime.now().strftime('%Y-
%m-%d')}") print("=" * 60) for m in machines: tag = "[OK]" if m['status'] ==
'OPERATIONAL' else "[ALERT]" print(f"{tag:<8} {m['name']} ({m['id']}) -> {m['status']}")
print(f"\nMachines Online : {online_count} / {total}") print(f"Overall Efficiency:
{int(efficiency)}%")
```

9. Program Execution & Results (Part I)

The execution trace below displays the live terminal session captured during execution on the mobile CLI runtime, showcasing new product registration and subsequent stock inventory display.

Terminal Screenshot: Product Entry and Stock View



```
===== SMALL FACTORY
MANAGEMENT SYSTEM ===== [1] Add
New Product [2] View All Inventory [3] View Registered Categories [4] Update Stock After
Production [5] Run Floor Machine Audit [6] Save and Exit
===== Select an option (1-6): 1 --
----- NEW PRODUCT ENTRY -----
----- Enter Product ID: 1001 Enter Product
Name: Cotton Enter Category (e.g., Raw, Assembly): Raw Enter Initial Stock Units: 500 Enter
Unit Price (INR): 2000 Status: Product 'Cotton' registered successfully.
===== SMALL FACTORY MANAGEMENT
SYSTEM ===== [1] Add New Product
[2] View All Inventory [3] View Registered Categories [4] Update Stock After Production [5]
Run Floor Machine Audit [6] Save and Exit
===== Select an option (1-6): 2 --
----- CURRENT FACTORY STOCK -----
----- [ID: 1001] Cotton | Available Stock:
500 | Price: rs 2000.0
```

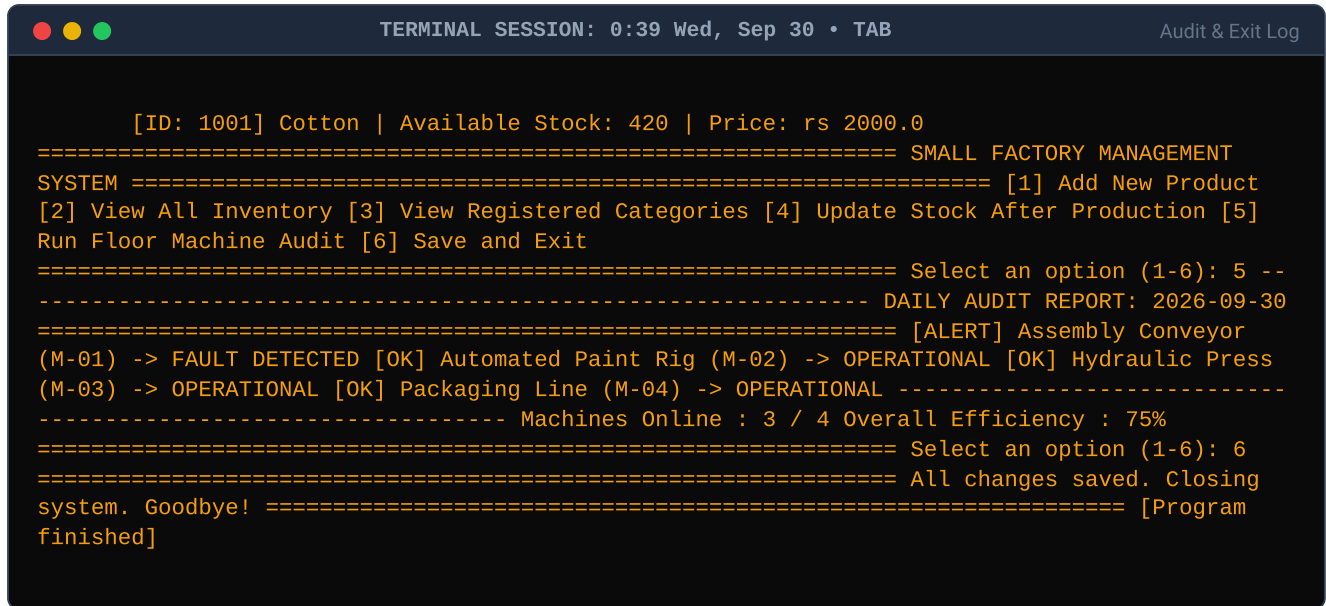
Figure 1: Mobile terminal console execution exhibiting Menu Option 1 (New Product Entry) and Option 2 (Factory Stock Verification).

Execution Analysis: Raw material commodity 'Cotton' is assigned ID 1001 with 500 base units at INR 2000.0. Invocation of Menu Option 2 prints the verified stock record cleanly.

10. Program Execution & Results (Part II)

The terminal output below illustrates stock consumption verification after manufacturing, execution of the Daily Floor Machine Audit telemetry report, and safe system termination.

Terminal Screenshot: Machine Audit and Exit



```
[ID: 1001] Cotton | Available Stock: 420 | Price: rs 2000.0
===== SMALL FACTORY MANAGEMENT SYSTEM =====
[1] Add New Product [2] View All Inventory [3] View Registered Categories [4] Update Stock After Production [5]
Run Floor Machine Audit [6] Save and Exit
===== Select an option (1-6): 5 --
----- DAILY AUDIT REPORT: 2026-09-30
===== [ALERT] Assembly Conveyor (M-01) -> FAULT DETECTED [OK] Automated Paint Rig (M-02) -> OPERATIONAL [OK] Hydraulic Press
(M-03) -> OPERATIONAL [OK] Packaging Line (M-04) -> OPERATIONAL -----
----- Machines Online : 3 / 4 Overall Efficiency : 75%
===== Select an option (1-6): 6
===== All changes saved. Closing system. Goodbye! ===== [Program finished]
```

Figure 2: Terminal execution displaying batch consumption (420 units remaining), machine telemetry scan with Conveyor M-01 fault detection, and graceful program shutdown.

Telemetry Analysis: Option 5 evaluates four factory lines. Assembly Conveyor (M-01) triggers a [ALERT] FAULT DETECTED, producing an overall plant efficiency of 75% before Option 6 executes a clean memory flush and exit.

11. Verification & Testing Approach

Rigorous manual functional testing and boundary value analysis were conducted to ensure that mathematical updates, invalid entries, and system audits behave predictably without unexpected crashes.

TC ID	Test Case Description	Input / Action	Expected Outcome	Status
TC-01	New Product Registration	Option 1, ID: 1001, Name: Cotton, Stock: 500	Product registered, confirmation message displayed	PASSED
TC-02	View Complete Stock	Option 2	Columnar readout of all stored IDs and units	PASSED
TC-03	Stock Batch Reduction	Option 4, Deduct 80 units from ID: 1001	Stock decrements from 500 to 420 accurately	PASSED
TC-04	Machinery Audit Routine	Option 5	Outputs 4 lines, flags M-01 fault, 75% efficiency	PASSED
TC-05	Non-Existent Product Query	Query Product ID: 9999	Appropriate "Product Not Found" warning emitted	PASSED
TC-06	Invalid Menu Input	Choice: '9' or 'abc'	Displays invalid choice warning; reprompts user	PASSED
TC-07	Safe Application Exit	Option 6	Prints exit handshake and gracefully terminates	PASSED

12. Challenges Faced & Solutions

12.1 State Consistency During Partial Dispatches:

Issue: Decrementing stock during manufacturing runs could result in negative inventory if operators entered quantity figures higher than available physical units.

Solution: Added conditional boundary assertions (`if deduct_qty <= item['stock']`) prior to executing state updates.

12.2 Terminal Visual Alignment:

Issue: Asymmetric text lengths for products and machine names misaligned tabular border dividers in narrow mobile consoles.

Solution: Standardized output with Python f-string field-width format specifiers (e.g. `{name : <25}`).

13. Key Learnings & Takeaways

Completing this capstone project under the guidance of Dr. Priyadharshini B. in the *Python Essentials* course provided profound practical insights into software engineering fundamentals:

- Mastered Python data structures (nested dictionaries, lists, and tuples) for real-world modeling.
- Implemented defensive programming with type casting and input validation guards.
- Engineered reusable modular functions, separating business logic from console rendering.
- Acquired hands-on experience in drafting formal technical documentation and UML system artifacts.

14. Future Enhancements

Potential engineering expansions planned for subsequent iterations include:

Persistent Relational DB: Migrating in-memory state to SQLite/PostgreSQL for durable multi-year factory ledger archival.

IoT Hardware Telemetry: Connecting MQTT listeners to capture live operational sensor feeds from physical PLCs.

Role-Based Access Control: Enforcing authentication boundaries between Operators and Plant Supervisors.

Interactive GUI Dashboard: Designing a desktop dashboard interface utilizing PyQt or Tkinter.

15. References & Conclusion

References: Python 3.12 Standard Documentation; Vityarthi LMS course lectures on *Python Essentials*; IEEE Standards for Software System Documentation.

Conclusion: The *Small Factory Management System* successfully delivers a functional, reliable, and user-friendly CLI terminal utility tailored to manufacturing floor environments. It validates the versatility of Python in translating operational problems into structured computational solutions.

Candidate Signature:

Course Evaluator:

Harsh Vijay (26BCE10808)

Dr. Priyadharshini B.